1. What are the main features of React?

Answer: The main features of React include:

1. JSX: A syntax extension that allows mixing HTML with JavaScript.
2. Virtual DOM: A lightweight copy of the actual DOM that helps improve performance.
3. Components: Reusable and self-contained pieces of code that define how parts of an application look and behave.
4. Unidirectional Data Flow: Ensures data flows in a single direction, making it easier to understand and debug.
5. Lifecycle Methods: Hooks into different stages of a component's lifecycle, such as mounting, updating, and unmounting.

2. Explain the concept of Virtual DOM and its benefits.

Answer: The Virtual DOM is a programming concept where a virtual representation of the UI is kept in memory and synced with the real DOM using a library like ReactDOM. Benefits include:

1. Improved performance: Updates are batched and efficient, minimizing direct manipulation of the real DOM.
2. Better user experience: Faster updates and rendering provide a smoother experience.
3. Predictable state management: Helps in managing the state of the application more predictably.

3. What is JSX?

Answer: JSX stands for JavaScript XML. It is a syntax extension for JavaScript that allows you to write HTML-like code within JavaScript. It makes the code more readable and easier to write. JSX is transpiled to React.createElement calls by tools like Babel.

4. What are components in React?

Answer: Components are the building blocks of a React application. They are reusable, self-contained pieces of code that define the structure, behavior, and rendering of parts of the UI. There are two main types of components:

1. Functional Components: Written as JavaScript functions.
2. Class Components: Written as ES6 classes.

5. What is the difference between state and props?

Answer:

State: A built-in object that holds data or information about the component. It is mutable and can be changed using the setState method. State is local to the component and is managed within it.

Props: Short for properties, props are read-only objects passed from parent to child components. They are used to pass data and event handlers down the component tree.

6. Explain the component lifecycle in React.

Answer: The React component lifecycle consists of:

Mounting: When a component is being inserted into the DOM.

1. Constructor()
2. static getDerivedStateFromProps()
3. render()
4. componentDidMount()

Updating: When a component is being re-rendered as a result of changes to props or state.

1. static getDerivedStateFromProps()
2. shouldComponentUpdate()

3. render()
4. getSnapshotBeforeUpdate()
5. componentDidUpdate()

Unmounting: When a component is being removed from the DOM.

1. componentWillUnmount()

Error Handling: When there is an error during rendering, in a lifecycle method, or in the constructor of any child component.

1. static getDerivedStateFromError()
2. componentDidCatch()

7. What are hooks in React?

Answer: Hooks are functions that let you use state and other React features in functional components. They provide a way to reuse stateful logic without changing the component hierarchy. Common hooks include:

1. useState: Allows using state in functional components.
2. useEffect: Performs side effects in functional components.
3. useContext: Accesses context in functional components.
4. useReducer: Manages complex state logic.
5. useMemo: Memoizes expensive calculations.

8. How does the useEffect hook work?

Answer: The useEffect hook lets you perform side effects in functional components. It takes two arguments: a function and an optional dependency array. The function runs after the first render and after every update if the dependencies change. If no dependencies are provided, it runs after every render. The function can return a cleanup

function to run before the component unmounts or before the effect re-runs.

9. What is the purpose of the useState hook?

Answer: The useState hook allows you to add state to functional components. It returns an array with two elements: the current state value and a function to update it. You can use it to manage component-level state without needing class components.

10. How can you optimize the performance of a React application?

Answer: Performance optimization techniques in React include:

1. Using the Virtual DOM efficiently.
2. Memoizing components using React.memo.
3. Avoiding unnecessary re-renders with shouldComponentUpdate or React.PureComponent.
4. Lazy loading components with React.lazy and Suspense.
5. Using the useMemo and useCallback hooks to memoize values and functions.
6. Code splitting with dynamic import().
7. Using the useDeferredValue and useTransition hooks for smooth rendering of complex UIs.

11. What is React Context and how do you use it?

Answer: React Context is a way to share values between components without passing props through every level of the tree. It consists of a Provider component that supplies the value and a Consumer component that accesses it. The useContext hook simplifies accessing context in functional components.

12. What is the difference between controlled and uncontrolled components in React?

Answer:

Controlled Components: Components where form data is handled by the React component. The component's state is the single source of truth for the input values.

Uncontrolled Components: Components where form data is handled by the DOM itself. Refs are used to access the input values from the DOM.

13. How does React handle forms?

Answer: React handles forms using controlled or uncontrolled components. Controlled components use state to manage input values and event handlers to update the state. Uncontrolled components use refs to access input values directly from the DOM.

14. What are refs in React?

Answer: Refs (short for references) provide a way to access and interact with DOM nodes or React elements directly. They are created using React.createRef() or the useRef hook and can be attached to elements via the ref attribute.

15. What is React Router?

Answer: React Router is a library for handling routing in React applications. It allows you to define routes and navigate between different views or components. Key components include BrowserRouter, Route, Link, Switch, and Redirect.

16. How do you handle side effects in a React component?

Answer: Side effects in React components are handled using the useEffect hook. The hook takes a function that performs the side effect and an optional dependency array to control when the effect

runs. The function can also return a cleanup function to handle cleanup tasks.

17. What are Higher-Order Components (HOC)?

Answer: Higher-Order Components (HOC) are functions that take a component and return a new component with additional props or functionality. They are used to reuse component logic and are a pattern for enhancing components.

18. What is the difference between useEffect and useLayoutEffect?

Answer:

1. useEffect: Runs after the render is committed to the screen. It is useful for side effects that don't block the browser paint, such as fetching data or subscribing to events.
2. useLayoutEffect: Runs synchronously after all DOM mutations and before the browser paints. It is useful for reading layout from the DOM and synchronously re-rendering.

19. How do you manage global state in a React application?

Answer: Global state in a React application can be managed using:

1. Context API: Shares state across the component tree.
2. State management libraries like Redux, MobX, or Recoil: Provide advanced state management solutions with features like time-travel debugging and middleware support.

20. What is Redux and how does it work with React?

Answer: Redux is a state management library that helps manage the global state of an application. It works with React by providing a predictable state container through the use of actions, reducers, and a single store. React-Redux, a binding library, connects Redux with React components.

21. Explain the concept of middleware in Redux.

Answer: Middleware in Redux is a way to extend Redux with custom functionality. It provides a third-party extension point between dispatching an action and the moment it reaches the reducer. Common use cases include logging, crash reporting, performing asynchronous tasks, and handling side effects.

22. What is the use of the connect function in React-Redux?

Answer: The connect function in React-Redux is used to connect React components to the Redux store. It provides a way to map state and dispatch to component props, allowing the component to read from the Redux store and dispatch actions.

23. How do you handle asynchronous actions in Redux?

Answer: Asynchronous actions in Redux are handled using middleware such as Redux Thunk or Redux Saga. Redux Thunk allows you to write action creators that return a function instead of an action, enabling you to perform asynchronous operations. Redux Saga uses generator functions to handle asynchronous tasks in a more declarative manner.

24. What are React Portals and when would you use them?

Answer: React Portals provide a way to render children into a DOM node that exists outside the parent component's DOM hierarchy. They are useful for rendering modals, tooltips, or any component that needs to visually break out of its parent container while still maintaining the React component tree structure.

25. How do you handle errors in a React application?

Answer: Errors in a React application can be handled using:

1. Error boundaries: Components that catch JavaScript errors in their child component tree and render a fallback UI.

2. The componentDidCatch lifecycle method or the static getDerivedStateFromError method.
3. Using try-catch blocks in synchronous code and promise catch handlers in asynchronous code.

26. What are PropTypes and why are they used in React?

Answer: PropTypes is a type-checking library used in React to validate the props passed to a component. It helps catch bugs by ensuring that components receive the correct data types. PropTypes can be used to specify required props, default values, and custom validation functions.

27. What is the difference between React and React Native?

Answer:

React: A JavaScript library for building web applications with reusable UI components.

React Native: A framework for building native mobile applications using React. It uses native components instead of web components and provides access to native APIs.

28. What is Server-Side Rendering (SSR) and how does it work in React?

Answer: Server-Side Rendering (SSR) is the process of rendering a React application on the server and sending the fully rendered HTML to the client. This improves the initial load time and SEO. Tools like Next.js provide built-in support for SSR in React applications.

29. What is Code Splitting and how do you implement it in React?

Answer: Code splitting is a technique to split a large bundle of code into smaller chunks that can be loaded on demand. It improves the performance of the application by reducing the initial load time. In

React, code splitting can be implemented using dynamic imports with React.lazy and Suspense.

30. What is the use of the useReducer hook?

Answer: The useReducer hook is an alternative to useState for managing complex state logic in functional components. It takes a reducer function and an initial state as arguments and returns the current state and a dispatch function. It is useful for handling state transitions based on specific actions.

31. Explain the concept of reconciliation in React.

Answer: Reconciliation is the process by which React updates the DOM to match the virtual DOM. When a component's state or props change, React creates a new virtual DOM tree and compares it to the previous one. This comparison, or diffing, allows React to determine the minimal set of changes needed to update the real DOM efficiently.

32. How do you handle routing in a React application?

Answer: Routing in a React application is handled using React Router. It provides components like BrowserRouter, Route, Link, Switch, and Redirect to define and manage routes, navigate between views, and handle route matching.

33. What is the difference between componentDidMount and useEffect?

Answer:

componentDidMount: A lifecycle method in class components that runs after the component is mounted to the DOM. It is used for side effects like fetching data or setting up subscriptions.

useEffect: A hook in functional components that can replicate the behavior of componentDidMount, componentDidUpdate, and componentWillUnmount. It runs after every render and can be controlled using the dependency array.

34. What are the advantages of using TypeScript with React?

Answer: Advantages of using TypeScript with React include:

1. Static type checking: Catches type-related errors at compile time.
2. Improved code quality and maintainability: Clearer type definitions and interfaces.
3. Better developer experience: Enhanced code completion, navigation, and refactoring support in IDEs.
4. Easier integration with existing JavaScript code: TypeScript is a superset of JavaScript.

35. What is the purpose of the key attribute in React?

Answer: The key attribute in React helps identify which items have changed, been added, or removed in a list of elements. It provides a way for React to optimize the rendering process by minimizing re-renders and ensuring efficient updates. Keys should be unique among siblings but stable and predictable.

36. Explain the concept of lifting state up in React.

Answer: Lifting state up is a technique in React where state is moved to the closest common ancestor of the components that need to share it. This allows multiple components to access and update the same state, ensuring data consistency and reducing duplication.

37. How do you handle authentication in a React application?

Answer: Authentication in a React application can be handled using:

1. Authentication libraries like Firebase, Auth0, or AWS Amplify.
2. Custom authentication logic with JWTs (JSON Web Tokens) and OAuth.
3. Storing authentication tokens in cookies or local storage.

4. Protecting routes using higher-order components or hooks like useAuth.

38. What is the use of the useCallback hook?

Answer: The useCallback hook memoizes a function, preventing it from being recreated on every render. It takes a function and a dependency array as arguments and returns a memoized version of the function. It is useful for optimizing performance, especially when passing callbacks to child components.

39. What is the purpose of the useMemo hook?

Answer: The useMemo hook memoizes a value, preventing it from being recalculated on every render. It takes a function that returns the value and a dependency array as arguments. It is useful for optimizing performance by avoiding expensive calculations and improving rendering efficiency.

40. How do you implement a custom hook in React?

Answer: A custom hook in React is a reusable function that encapsulates stateful logic. It starts with the prefix “use” and can use other hooks. To implement a custom hook, create a function that uses hooks like useState or useEffect and returns state and/or functions to be used by components.

41. What is the role of the shouldComponentUpdate lifecycle method?

Answer: The shouldComponentUpdate lifecycle method is used to control whether a component should re-render when its state or props change. It returns a boolean value: true to allow the update and false to prevent it. It is useful for optimizing performance by avoiding unnecessary re-renders.

42. Explain the concept of PureComponent in React.

Answer: PureComponent is a base class in React that provides a shallow comparison of props and state to determine if a component should re-render. It automatically implements the shouldComponentUpdate method with a shallow comparison, optimizing performance by preventing unnecessary re-renders.

43. What is the difference between useState and useReducer?

Answer:

useState: A hook for adding state to functional components. It is simple and straightforward, suitable for managing simple state logic.

useReducer: A hook for managing complex state logic. It uses a reducer function to handle state transitions based on actions, providing a more scalable and organized way to manage state.

44. How do you handle forms in React using Formik?

Answer: Formik is a library for building and managing forms in React. It provides components and hooks to handle form state, validation, and submission. Formik simplifies form handling by providing built-in support for common form tasks like managing input values, validating inputs, and handling form submissions.

45. What is the Context API in React?

Answer: The Context API is a feature in React that allows you to share values between components without passing props through every level of the component tree. It consists of a Provider component that supplies the value and a Consumer component or the useContext hook to access the value.

46. How do you implement lazy loading in React?

Answer: Lazy loading in React can be implemented using the React.lazy function and the Suspense component. React.lazy allows you to dynamically import components, splitting the code and loading

it only when needed. The Suspense component provides a fallback UI while the lazy-loaded component is being fetched.

47. What is the purpose of the useRef hook?

Answer: The useRef hook provides a way to create a mutable reference that persists across re-renders. It can be used to access and interact with DOM elements directly or to store mutable values that do not trigger re-renders when changed.

48. How do you handle animations in a React application?

Answer: Animations in a React application can be handled using:

1. CSS transitions and animations.
2. Libraries like React Transition Group, Framer Motion, or React Spring.
3. The use of the requestAnimationFrame API for custom animations.

49. What is the difference between class components and functional components in React?

Answer:

Class Components: Written as ES6 classes, support lifecycle methods, and manage state using the this.state object and setState method.

Functional Components: Written as JavaScript functions, initially stateless but can use hooks like useState and useEffect to manage state and lifecycle.

50. What are the benefits of using hooks in React?

Answer: Benefits of using hooks in React include:

1. Simplified component logic by using functions instead of classes.
2. Improved code readability and maintainability.

3. Reusable stateful logic with custom hooks.
4. Easier to understand and test components.
5. Reduced boilerplate code.

51. How do you implement internationalization (i18n) in a React application?

Answer: Internationalization (i18n) in a React application can be implemented using libraries like react-i18next or react-intl. These libraries provide tools and components to manage translations, handle locale changes, and format dates, numbers, and currencies based on the user's locale.

52. What is the difference between shallow and deep rendering in testing React components?

Answer:

Shallow Rendering: Renders a component without rendering its children. It is useful for unit testing a single component in isolation.

Deep Rendering: Renders a component along with all its children. It is useful for integration testing to ensure components work together as expected.

53. How do you test React components using Jest and Enzyme?

Answer: Testing React components using Jest and Enzyme involves:

1. Writing test cases using Jest's test, expect, and other assertion functions.
2. Using Enzyme's shallow, mount, or render functions to render components.
3. Asserting the component's output, state, and behavior using Enzyme's API and Jest's matchers.

54. What is the purpose of the getSnapshotBeforeUpdate lifecycle method?

Answer: The getSnapshotBeforeUpdate lifecycle method is called right before the DOM is updated. It allows you to capture information from the DOM, such as scroll position, before the update occurs. The value returned from this method is passed as an argument to the componentDidUpdate method.

55. How do you implement server-side rendering with Next.js?

Answer: Server-side rendering with Next.js is implemented using:

1. The getServerSideProps function to fetch data on the server for each request.
2. The getInitialProps method for data fetching in class components or custom _app.js and _document.js files.
3. Automatic static optimization and dynamic routing features provided by Next.js.

56. What is the difference between client-side rendering and server-side rendering?

Answer:

Client-Side Rendering (CSR): The application is rendered in the browser using JavaScript. The initial load is faster, but the user sees a loading spinner or blank page while the JavaScript is fetched and executed.

Server-Side Rendering (SSR): The application is rendered on the server, and the fully rendered HTML is sent to the client. The initial load is slower, but the user sees the content immediately, improving perceived performance and SEO.

57. How do you handle state management in a large-scale React application?

Answer: State management in a large-scale React application can be handled using:

1. The Context API for simple global state.
2. State management libraries like Redux, MobX, or Recoil for more complex state logic.
3. Breaking down the state into local and global state and using hooks like useState, useReducer, and useContext.

58. What is the role of selectors in Redux?

Answer: Selectors in Redux are functions that extract and compute derived data from the Redux store. They provide a way to encapsulate complex state selection logic, improve performance by memoizing results with libraries like Reselect, and ensure components only receive the data they need.

59. How do you implement caching in a React application?

Answer: Caching in a React application can be implemented using:

1. Browser APIs like localStorage, sessionStorage, and IndexedDB.
2. Service workers for caching assets and API responses.

3. Libraries like SWR or React Query for caching and synchronizing server state with the client.

60. What is the purpose of the useImperativeHandle hook?

Answer: The useImperativeHandle hook customizes the instance value that is exposed to parent components when using refs. It allows you to control which methods and properties are accessible, providing a more controlled way to interact with the component's internal state and methods.

61. How do you handle complex animations in React?

Answer: Complex animations in React can be handled using:

1. CSS animations and transitions for simple animations.
2. Libraries like Framer Motion, React Spring, or GSAP for more complex animations.
3. The use of the requestAnimationFrame API for custom animations.

62. What is the purpose of the useDebugValue hook?

Answer: The useDebugValue hook is used to display a label in React DevTools for custom hooks. It provides a way to add custom debugging information for hooks, making it easier to understand and debug the state and behavior of custom hooks.

63. How do you implement optimistic updates in a React application?

Answer: Optimistic updates in a React application can be implemented by:

1. Updating the UI immediately after an action, before the server confirms the change.
2. Reverting the UI if the server request fails.
3. Using state management libraries like Redux or React Query to manage the optimistic state and handle success and failure scenarios.

64. What is the purpose of the Suspense component in React?

Answer: The Suspense component in React is used to handle the loading state of asynchronous components. It allows you to specify a fallback UI to be displayed while the lazy-loaded component or data is being fetched. Suspense simplifies handling asynchronous operations and improves user experience.

65. How do you handle code splitting in a React application?

Answer: Code splitting in a React application can be handled using:

1. Dynamic imports with React.lazy and Suspense for component-level splitting.
2. Tools like Webpack or Rollup for bundling and splitting code.
3. Libraries like Loadable Components for more advanced code-splitting and server-side rendering support.

66. What are error boundaries and how do you implement them in React?

Answer: Error boundaries are React components that catch JavaScript errors in their child component tree and render a fallback UI. They are implemented using the static getDerivedStateFromError and componentDidCatch lifecycle methods. Error boundaries help prevent crashes and improve user experience.

67. How do you manage side effects in a Redux application?

Answer: Side effects in a Redux application can be managed using middleware like Redux Thunk or Redux Saga. Redux Thunk allows you to write action creators that return a function for handling side effects. Redux Saga uses generator functions to handle complex asynchronous logic in a more declarative manner.

68. What is the purpose of the createSelector function in Reselect?

Answer: The createSelector function in Reselect is used to create memoized selectors for Redux state. It allows you to define functions that compute derived state, improving performance by caching the results of expensive calculations and preventing unnecessary re-renders.

69. How do you implement drag-and-drop functionality in a React application?

Answer: Drag-and-drop functionality in a React application can be implemented using:

1. The HTML5 Drag and Drop API.
2. Libraries like react-dnd or react-beautiful-dnd for more advanced and customizable drag-and-drop interactions.

70. What is the purpose of the useTransition hook in React?

Answer: The useTransition hook is used to manage transitions between different states in a React application. It allows you to specify a pending state while a transition is in progress, improving user experience by providing feedback during complex state changes or asynchronous operations.

71. How do you handle accessibility in a React application?

Answer: Handling accessibility in a React application involves:

1. Using semantic HTML elements.
2. Adding ARIA (Accessible Rich Internet Applications) attributes to improve screen reader support.
3. Ensuring keyboard navigation and focus management.
4. Using tools like React A11y, axe-core, or the Lighthouse accessibility audit.

72. What is the purpose of the useDeferredValue hook in React?

Answer: The useDeferredValue hook is used to defer updating a value until after the current render. It allows you to prioritize rendering updates and avoid blocking the main thread with expensive calculations, improving performance and user experience in complex UIs.

73. How do you handle conditional rendering in React?

Answer: Conditional rendering in React can be handled using:

1. Ternary operators or logical && operators to conditionally render elements.
2. Conditional expressions inside JSX.
3. Returning null to render nothing.
4. Using helper functions or components for more complex conditions.

74. What is the purpose of the StrictMode component in React?

Answer: The StrictMode component in React is a tool for highlighting potential problems in an application. It activates additional checks and warnings for its descendants in development mode, helping you identify and fix issues related to deprecated APIs, side effects, and more.

75. How do you handle forms in React using React Hook Form?

Answer: React Hook Form is a library for handling forms in React. It provides a way to manage form state, validation, and submission using hooks. It simplifies form handling by offering features like uncontrolled inputs, custom validation, and minimal re-renders.

76. What is the purpose of the useMutation hook in React Query?

Answer: The useMutation hook in React Query is used to handle asynchronous operations that modify data, such as creating, updating, or deleting records. It provides functions to trigger the mutation, manage loading and error states, and handle side effects like invalidating queries or updating the cache.

77. How do you implement virtual scrolling in a React application?

Answer: Virtual scrolling in a React application can be implemented using libraries like react-window or react-virtualized. These libraries

render only the visible portion of a large list, improving performance by reducing the number of DOM elements.

78. What is the purpose of the useInfiniteQuery hook in React Query?

Answer: The useInfiniteQuery hook in React Query is used to handle paginated data fetching. It provides functions to fetch more data, manage loading and error states, and merge the results, making it easier to implement infinite scrolling or load more patterns.

79. How do you handle error boundaries in React functional components?

Answer: Error boundaries are typically implemented in class components. However, in functional components, you can use hooks like useErrorBoundary from libraries like react-error-boundary to provide similar error handling capabilities.

80. What is the purpose of the useLayoutEffect hook in React?

Answer: The useLayoutEffect hook is similar to useEffect but runs synchronously after all DOM mutations. It is useful for reading layout information or synchronously re-rendering before the browser paints the screen, ensuring the layout is updated before the user sees it.

Part 2

1. What is the significance of React Fragments?

Answer: React Fragments allow you to group multiple elements without adding unnecessary nodes to the DOM. It’s useful when you don’t want to introduce an additional parent div.

2. What are React Hooks?

Answer: React Hooks are functions that let you use state and other React features in functional components instead of class components.

3. Explain the useEffect Hook.

Answer: useEffect is a Hook that performs side effects in your function components. It runs after every render and can be used for data fetching, subscriptions, manually changing the DOM, etc.

4. What is the purpose of the useState Hook?

Answer: The useState Hook in React is used to add state variables to functional components, allowing them to keep track of state changes and trigger re-renders.

5. What is the significance of keys in React lists?

Answer: Keys are used to give React a way to identify which items have changed, are added, or are removed in a list. They help in optimizing the rendering process.

6. What is the Context API in React?

Answer: The Context API provides a way to pass data through the component tree without having to pass props down manually at every level.

7. How does React handle prop drilling, and what's the solution?

Answer: Prop drilling occurs when you pass props through multiple levels of components. The solution is to use Context API or Redux for managing state globally.

8. What is Redux, and why might you use it with React?

Answer: Redux is a predictable state container for JavaScript apps. It's often used with React to manage the state of the application in a central store, making state management more predictable and easier to debug.

9. What is the significance of PureComponent in React?

Answer: PureComponent is a base class for class components that implements a shouldComponentUpdate method with a shallow prop

and state comparison. It helps in preventing unnecessary renders for performance optimization.

10. Explain React Router.

Answer: React Router is a standard library for routing in React applications. It enables the navigation among views in the application, allowing the user to interact with the UI without a full page reload.

11. What is the significance of the memo function in React?

Answer: The memo function in React is used to memoize functional components, preventing unnecessary re-renders if the props haven't changed.

12. What is the purpose of the useReducer Hook?

Answer: useReducer is a Hook in React used for state management. It is preferable for managing complex state logic that involves multiple sub-values or when the next state depends on the previous one.

13. Explain the concept of higher-order components (HOCs) in React.

Answer: Higher-order components are functions that take a component and return a new component with extended or modified functionality. They are used for code reuse, logic abstraction, and component composition.

14. What is the role of the key attribute in React?

Answer: The key attribute is used to uniquely identify elements in a React list. It helps React identify which items have changed, been added, or been removed.

15. How does React handle forms?

Answer: React handles forms using controlled components, where the form elements like input fields are controlled by the state. This allows React to manage and validate the form data.

16. What is Redux Thunk, and why might you use it?

Answer: Redux Thunk is a middleware for Redux that allows you to write asynchronous logic in action creators. It is commonly used for handling side effects, such as making asynchronous API calls.

17. What is the purpose of the componentWillUnmount lifecycle method in class components?

Answer: componentWillUnmount is called just before a component is unmounted and destroyed. It is often used for cleanup operations, such as canceling network requests or clearing up subscriptions.

18. What is the significance of the forwardRef function in React?

Answer: forwardRef is used for forwarding refs to child components in React. It's particularly useful when you're using higher-order components or when dealing with third-party libraries that rely on refs.

19. What are React Hooks rules?

Answer: Hooks in React must be called at the top level (not inside loops or conditions), and they must be called from React functional components or custom Hooks.

20. Explain the concept of server-side rendering (SSR) in React.

Answer: Server-side rendering in React involves rendering the initial HTML on the server rather than the browser. This can improve the page load performance and provide better SEO.

21. What is the purpose of the React.StrictMode component?

Answer: React.StrictMode is a component in React used for highlighting potential problems in the application during the development mode. It helps catch common mistakes and future proofs the code.

22. How does React handle routing?

Answer: React uses libraries like React Router for handling routing in applications. It enables the mapping of components to specific routes, allowing for a single-page application experience.

23. What is the significance of the dangerouslySetInnerHTML attribute in React?

Answer: dangerouslySetInnerHTML is used to inject HTML content into a React component. It should be used with caution, as improper use can expose the application to Cross-Site Scripting (XSS) attacks.

24. Explain the concept of controlled components in React.

Answer: Controlled components in React are components where the value of form elements like input fields is controlled by the state. This allows React to handle and validate the form data.

25. What is the purpose of the useContext Hook in React?

Answer: useContext is a Hook in React used for consuming the value of a context. It provides a way to access the values of a context directly within a functional component.

26. What is the significance of the shouldComponentUpdate method in React?

Answer: shouldComponentUpdate is a lifecycle method in class components that determines whether the component should re-render. It can be used to optimize performance by preventing unnecessary renders.

27. Explain the concept of code-splitting in React.

Answer: Code-splitting is a technique in React that involves breaking the code into smaller chunks and loading them on demand. This helps in optimizing the initial loading time of the application.

28. How does React handle events?

Answer: React uses a synthetic event system that normalizes events across different browsers. Event handlers in React are named using camelCase, such as onClick instead of onclick.

29. What is the purpose of the useMemo Hook in React?

Answer: The useMemo Hook is used to memoize the result of a computation. It helps in optimizing performance by preventing unnecessary recalculations.

30. Explain the concept of portals in React.

Answer: Portals in React provide a way to render children into a DOM node that exists outside the DOM hierarchy of the parent component. This can be useful for creating overlays or modals.

31. What is the purpose of the React.Fragment element?

Answer: React.Fragment is a built-in component in React that allows you to group multiple children without introducing an additional DOM element.

32. Explain the concept of Redux middleware.

Answer: Middleware in Redux provides a third-party extension point between dispatching an action and the moment it reaches the reducer. It is often used for logging, asynchronous actions, and other side effects.

33. What is the significance of the useLayoutEffect Hook in React?

Answer: useLayoutEffect is similar to useEffect but fires synchronously after all DOM mutations. It is often used when you need to read from the DOM immediately after an update.

34. How does React handle errors in components?

Answer: React components can catch JavaScript errors anywhere in their tree using the error boundaries concept. This is done using the componentDidCatch lifecycle method.

35. Explain the concept of the key prop in React lists.

Answer: The key prop is used to give a unique identifier to elements in a list. It helps React efficiently update the UI by reusing existing elements and identifying changes.

36. What is the purpose of the useCallback Hook in React?

Answer: useCallback is used to memoize callback functions, preventing unnecessary re-renders of child components that rely on these callbacks.

37. How does React handle conditional rendering?

Answer: Conditional rendering in React is achieved using JavaScript expressions. You can use if statements or ternary operators within JSX to conditionally render components.

38. What is the role of the displayName property in React?

Answer: The displayName property is used to set a display name for a React component, making it easier to debug and profile the component hierarchy.

39. Explain the concept of hooks rules in React.

Answer: Rules for using hooks in React include calling them at the top level, only calling them from functional components or custom hooks, and following the naming convention (e.g., starting with “use”).

40. What is the significance of the useImperativeHandle Hook?

Answer: useImperativeHandle is used to customize the instance value that is exposed when using React.forwardRef. It allows controlling what values are exposed externally.

41. What is the purpose of the contextType property in class components?

Answer: The contextType property in class components is used to consume the value of a context and make it available as this.context within the component.

42. Explain the concept of lazy loading in React.

Answer: Lazy loading is a technique where components or resources are loaded only when they are actually needed. React provides the React.lazy function for lazy loading components.

43. What is the significance of the useEffect cleanup function?

Answer: The cleanup function in the useEffect Hook is used to perform cleanup tasks (unsubscribe, clear timers, etc.) when the component unmounts or when the dependencies change.

44. How does React handle state in class components compared to functional components?

Answer: Class components use the this.state approach for managing state, while functional components use the useState Hook to manage state variables.

45. What is the role of the React.memo function in functional components?

Answer: React.memo is a higher-order component that memoizes the rendering of a functional component, preventing unnecessary renders if the props haven’t changed.

46. Explain the purpose of the useEffect dependency array.

Answer: The dependency array in the useEffect Hook is used to specify the values from the component’s scope that the effect depends on. The effect is re-run only if these values change.

47. What is the significance of the useRef Hook in React?

Answer: useRef is used to create mutable object properties that persist across renders without causing re-renders when they change. It is often used to access or modify the properties of DOM elements.

48. How does React handle forms in controlled components?

Answer: In controlled components, the form elements like input fields are controlled by the component's state. This allows React to manage and validate the form data.

49. What is the purpose of the React.PureComponent class in React?

Answer: React.PureComponent is similar to React.Component but implements shouldComponentUpdate with a shallow prop and state comparison, optimizing performance by preventing unnecessary renders.

50. Explain the concept of hooks in React.

Answer: Hooks are functions that enable functional components in React to have state, lifecycle methods, and other features that were previously only available in class components.

51. What is the role of the useReducer Hook in React?

Answer: useReducer is a Hook used for state management in React. It is often preferable for managing complex state logic that involves multiple sub-values or when the next state depends on the previous one.

52. Explain the concept of the React.StrictMode component in React.

Answer: React.StrictMode is a development mode tool in React that highlights potential problems in the application during development. It enables additional checks and warnings for common mistakes.

53. How does React handle uncontrolled components?

Answer: Uncontrolled components in React are those where the form data is handled by the DOM itself, rather than being controlled by React state. Refs are typically used to interact with uncontrolled components.

54. What is the significance of the data-testid attribute in React testing?

Answer: The data-testid attribute is often used in testing to provide a reliable and non-visual way of selecting elements in the DOM. It helps create robust and maintainable tests.

55. Explain the concept of the useEffect Hook dependencies array.

Answer: The dependencies array in the useEffect Hook contains variables from the component's scope that the effect depends on. The effect is re-run only if these dependencies change.

56. What is the purpose of the withRouter higher-order component in React Router?

Answer: withRouter is a higher-order component provided by React Router. It passes the history, location, and match props to the wrapped component, allowing access to the router's information.

57. How does React handle event propagation?

Answer: React uses a synthetic event system that normalizes event handling across different browsers. Event handlers in React use camelCase, such as onClick instead of onclick.

58. What is the significance of the useDebugValue Hook in React?

Answer: useDebugValue is a Hook used to display a label for custom hooks in React DevTools. It can be useful for providing additional information about the custom hook during development.

59. Explain the concept of the useLayoutEffect Hook in React.

Answer: useLayoutEffect is similar to useEffect but fires synchronously after all DOM mutations. It is often used when you need to read from the DOM immediately after an update.

60. What is the purpose of the dangerouslySetInnerHTML prop in React?

Answer: dangerouslySetInnerHTML is used to inject HTML content into a React component. It should be used with caution to prevent XSS vulnerabilities.

61. How does React handle routing in a single-page application?

Answer: React uses libraries like React Router for handling routing in single-page applications. It allows the mapping of components to specific routes, enabling a seamless user experience.

62. What is the significance of the React.createRef method in React?

Answer: React.createRef is used to create a mutable object that persists across renders and can be attached to React elements. It is often used to interact with the DOM or with React components directly.

63. Explain the concept of code splitting in React.

Answer: Code splitting is a technique in React that involves breaking the code into smaller chunks and loading them on demand. It helps optimize the initial loading time of the application.

64. What is the purpose of the forwardRef function in React?

Answer: forwardRef is used for forwarding refs to child components in React. It’s particularly useful when dealing with higher-order components or when integrating with third-party libraries.

65. How does React handle state management in functional components?

Answer: React introduced Hooks to enable state management in functional components. The useState Hook is commonly used to add state variables to functional components.

66. What is the significance of the useEffect Hook in React?

Answer: useEffect is a Hook in React used for handling side effects in functional components. It runs after every render and is commonly used for data fetching, subscriptions, and manual DOM manipulations.

67. Explain the concept of the useMemo Hook in React.

Answer: useMemo is used to memoize the result of a computation in React. It helps optimize performance by preventing unnecessary recalculations of values that don't change frequently.

68. What is the purpose of the React.Fragment component in React?

Answer: React.Fragment is a built-in component in React used to group multiple elements without introducing an additional DOM node. It helps keep the HTML structure clean.

69. Explain the concept of the useReducer Hook in React.

Answer: useReducer is a Hook in React used for state management. It is particularly useful for managing complex state logic that involves multiple sub-values or when the next state depends on the previous one.

70. What is the role of the useCallback Hook in React?

Answer: useCallback is used to memoize callback functions in React. It helps optimize performance by preventing unnecessary re-renders of components that rely on these callbacks.

71. How does React handle component lifecycle methods in functional components?

Answer: React introduced the useEffect Hook to handle side effects and mimic the lifecycle methods in functional components. It covers aspects like componentDidMount, componentDidUpdate, and componentWillUnmount.

72. What is the purpose of the useRef Hook in React?

Answer: useRef is used to create mutable object properties in React that persist across renders without causing re-renders when they change. It is often used to access or modify the properties of DOM elements.

73. Explain the concept of controlled and uncontrolled components in React.

Answer: Controlled components have their form data handled by React state, while uncontrolled components handle their form data by the DOM itself. Refs are often used to interact with uncontrolled components.

74. What is the significance of the useContext Hook in React?

Answer: useContext is a Hook in React used for consuming the value of a context. It provides a way to access the values of a context directly within a functional component.

75. How does React handle prop drilling, and what are potential solutions?

Answer: Prop drilling occurs when you pass props through multiple levels of components. Solutions include using the Context API, Redux, or custom Hooks to manage and share state globally.

76. What is the role of the useLayoutEffect Hook in React?

Answer: useLayoutEffect is similar to useEffect but fires synchronously after all DOM mutations. It is often used when you need to read from the DOM immediately after an update.

77. Explain the concept of the dangerouslySetInnerHTML attribute in React.

Answer: dangerouslySetInnerHTML is used to inject HTML content into a React component. It should be used with caution, as improper use can expose the application to Cross-Site Scripting (XSS) attacks.

78. What is the purpose of the React.StrictMode component in React?

Answer: React.StrictMode is a development mode tool in React that highlights potential problems in the application during development. It enables additional checks and warnings for common mistakes.

79. How does React handle error boundaries, and what is their purpose?

Answer: Error boundaries in React catch JavaScript errors anywhere in their component tree and log those errors, preventing the entire application from crashing. They are used for better error handling and logging.

80. What is the role of the useDebugValue Hook in React?

Answer: useDebugValue is a Hook used to display a label for custom hooks in React DevTools. It can be useful for providing additional information about the custom hook during development.

Top 30 Interview Questions and Answers for Senior Web Developers with React.js

1. What are the key differences between React.js and other JavaScript frameworks like Angular and Vue?

Answer:

- **React.js** is a library, not a full-fledged framework. It focuses on building user interfaces through a component-based

architecture. It uses a Virtual DOM for optimizing updates and employs JSX to make the code more readable.

- **Angular** is a full MVC framework that handles more of the app lifecycle and includes features like two-way data binding.
- **Vue.js** also uses a Virtual DOM but is more flexible with a gentle learning curve compared to Angular. It is similar to React in being component-based but offers some built-in features that React doesn't (e.g., a built-in state management solution).

2. Explain how the Virtual DOM works in React.js.

Answer: The Virtual DOM is an in-memory representation of the actual DOM. When a state change occurs in a React component, React first updates the Virtual DOM and then compares it with the previous Virtual DOM (using a process called "diffing"). React only applies the differences to the real DOM, optimizing updates.

```
function App() {
  const [count, setCount] = React.useState(0);

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => setCount(count +
1)}>Increment</button>
    </div>
  );
}
```

React only updates the <h1> tag in the real DOM, not the entire component.

3. What are React Hooks, and why were they introduced?

Answer: Hooks were introduced to allow developers to use state and other React features in functional components without needing to convert them into class components. Common hooks include useState, useEffect, useContext, etc.

```
function Counter() {
  const [count, setCount] = React.useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count +
1)}>Increment</button>
    </div>
  );
}
```

4. How do you manage state in React applications?

Answer: State can be managed in several ways:

- **Local State:** Managed via useState.
- **Global State:** Managed using Context API or state management libraries like Redux or MobX.

```
// Local state example:
const [count, setCount] = useState(0);

// Context API for global state:
const AppContext = React.createContext();
```

5. Explain the Context API and its typical use cases.

Answer: The Context API provides a way to pass data through the component tree without passing props manually at every level. It’s

useful for global data like user authentication, theme preferences, or language settings.

```
const ThemeContext = React.createContext('light');

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  return <ThemedButton />;
}

function ThemedButton() {
  const theme = React.useContext(ThemeContext);
  return <button className={theme}>Button</button>;
}
```

6. How does React's component lifecycle work, and how has it changed with the introduction of hooks?

Answer: Class components have lifecycle methods like componentDidMount, componentDidUpdate, and componentWillUnmount. Hooks such as useEffect replace these lifecycle methods in functional components.

```
function Component() {
  useEffect(() => {
    // componentDidMount or componentDidUpdate logic
    return () => {
      // componentWillUnmount logic
```

```
    };
  }, []);
}
```

7. What are the differences between controlled and uncontrolled components in React?

Answer:

- **Controlled components** rely on React state to manage form inputs.
- **Uncontrolled components** rely on the DOM and use refs for accessing values.

```
// Controlled component example:
function ControlledInput() {
  const [value, setValue] = React.useState('');

  return <input value={value} onChange={(e) =>
setValue(e.target.value)} />;
}

// Uncontrolled component example:
function UncontrolledInput() {
  const inputRef = React.useRef();

  return <input ref={inputRef} />;
}
```

8. How would you optimize the performance of a React application?

Answer:

- **Code Splitting:** Using React.lazy and Suspense.
- **Memoization:** Using React.memo, useMemo, and useCallback.

- **Avoid unnecessary re-renders:** Proper use of keys and preventing state mutation.
- **Lazy Loading:** Load components and data only when needed.

```
const MemoizedComponent = React.memo(({ count }) => {
  return <div>{count}</div>;
});
```

9. What is Prop Drilling, and how can it be avoided in React?

Answer: Prop drilling occurs when data is passed through multiple components that don't need it, just to reach a child component. This can be avoided by using the Context API or a state management library.

```
// Avoiding prop drilling using Context API
const UserContext = React.createContext();
```

10. What is the significance of keys in React, and how do they work in lists?

Answer: Keys help React identify which items in a list have changed, are added, or are removed, thus optimizing re-renders. Keys should be unique and stable.

```
<ul>
  {items.map(item => <li key={item.id}>{item.name}</li>)}
</ul>
```

11. How do you handle side effects in React components?

Answer: Side effects such as data fetching or subscriptions are handled using the useEffect hook.

```
useEffect(() => {
  // Data fetching or subscription
  return () => {
    // Clean-up subscription if necessary
```

```
  };
}, []); // Empty dependency array to run only on mount
```

12. What are Higher-Order Components (HOCs) in React, and when should you use them?

Answer: HOCs are functions that take a component and return a new component, useful for reusing component logic.

```
function withLoading(Component) {
  return function LoadingWrapper(props) {
    if (props.isLoading) return <div>Loading...</div>;
    return <Component {...props} />;
  };
}
```

13. What is React.memo, and how does it help with performance optimization?

Answer: React.memo is a higher-order component that prevents a functional component from re-rendering if its props haven't changed.

```
const MyComponent = React.memo(({ name }) => {
  return <div>{name}</div>;
});
```

14. Can you explain the difference between functional and class components?

Answer:

- **Class Components** have lifecycle methods and this keyword.
- **Functional Components** are simpler, use hooks like useState and useEffect, and are preferred in modern React development.

```
// Class component example:
class MyComponent extends React.Component {
```

```
  state = { count: 0 };
  render() {
    return <div>{this.state.count}</div>;
  }
}

// Functional component example:
function MyComponent() {
  const [count, setCount] = React.useState(0);
  return <div>{count}</div>;
}
```

15. What are Fragments in React, and why are they useful?

Answer: Fragments allow grouping multiple elements without adding extra nodes to the DOM, helping to avoid unnecessary wrapping elements like .

```
return (
  <React.Fragment>
    <h1>Title</h1>
    <p>Paragraph</p>
  </React.Fragment>
);
```

16. How would you handle form validation in React?

Answer: Form validation can be handled manually or using libraries like Formik or React Hook Form.

```
// Manual validation example:
function Form() {
  const [value, setValue] = React.useState('');
  const handleSubmit = (e) => {
    e.preventDefault();
    if (value === '') {
```

```
      alert('Field is required');
    }
  };
  return (
    <form onSubmit={handleSubmit}>
      <input value={value} onChange={(e) =>
setValue(e.target.value)} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

17. Explain React Router and how routing works in single-page applications.

Answer: React Router is used to manage routing in React apps. It allows developers to build single-page applications (SPAs) that navigate between different views without full page reloads.

```
import { BrowserRouter as Router, Route, Switch } from 'react-
router-dom';

function App() {
  return (
    <Router>
      <Switch>
        <Route path="/home" component={Home} />
        <Route path="/about" component={About} />
      </Switch>
    </Router>
  );
}
```

18. What is Redux, and how does it help manage state in large applications?

Answer: Redux is a state management library that helps manage the application state in a predictable manner. It follows a unidirectional data flow model and stores the global state in a single source of truth (store).

```
const store = createStore(reducer);

function reducer(state = initialState, action) {
  switch (action.type) {
    case 'INCREMENT':
      return { ...state, count: state.count + 1 };
    default:
      return state;
  }
}
```

19. What is the difference between Redux and Context API?

Answer:

- **Redux** is more suitable for complex state management and provides a more structured way to handle actions and state changes.
- **Context API** is simpler but might not scale well in very large applications with many layers of components.

20. How would you handle asynchronous actions in Redux?

Answer: Asynchronous actions are handled using middleware like Redux Thunk or Redux Saga.

```
// Redux Thunk example:
function fetchData() {
  return async (dispatch) => {
    const response = await fetch('url');
    dispatch({ type: 'FETCH_SUCCESS', payload: response.data });
```

```
  };
}
```

21. What is Server-Side Rendering (SSR) in React, and what are its benefits?

Answer: SSR renders React components on the server and sends the fully rendered HTML to the client. It improves SEO and speeds up the initial load.

22. Explain the concept of code splitting in React.

Answer: Code splitting allows the app to load only the necessary JavaScript for the page being viewed, reducing the initial load time. This can be achieved with React.lazy and Suspense.

```
const OtherComponent = React.lazy(() =>
import('./OtherComponent'));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <OtherComponent />
    </Suspense>
  );
}
```

23. How does React handle accessibility (a11y)?

Answer: React follows the same accessibility rules as regular HTML. You can use the aria- attributes to improve accessibility and also tools like the React A11y plugin to audit accessibility.

24. What are error boundaries in React?

Answer: Error boundaries are React components that catch JavaScript errors in their child component tree, log those errors, and display a fallback UI.

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong.</h1>;
    }

    return this.props.children;
  }
}
```

25. What is the difference between useEffect **and** useLayoutEffect**?**

Answer:

- **useEffect:** Runs asynchronously after the render.
- **useLayoutEffect:** Runs synchronously after the DOM mutations but before the browser repaints.

26. How do you handle code reusability in React?

Answer: Code reusability in React is handled using:

- **Higher-Order Components (HOCs)**
- **Custom Hooks**
- **Render Props**

27. How would you approach migrating a class-based component to a functional component with hooks?

Answer:

1. Identify the stateful logic and lifecycle methods.
2. Replace state and setState with useState.
3. Replace lifecycle methods (componentDidMount, componentDidUpdate, componentWillUnmount) with useEffect.

28. How does React handle security vulnerabilities like XSS attacks?

Answer: React automatically escapes any values embedded in JSX before rendering them, preventing XSS attacks. However, using dangerouslySetInnerHTML can open your app to XSS vulnerabilities.

```
const safeHTML = '<p>Safe HTML</p>';
<div dangerouslySetInnerHTML={{ __html: safeHTML }} />;
```

29. What are React portals, and when would you use them?

Answer: Portals provide a way to render children into a DOM node outside the parent component hierarchy. Useful for modals, tooltips, etc.

```
ReactDOM.createPortal(
  <ModalContent />,
  document.getElementById('modal-root')
);
```

30. How do you handle testing in React applications?

Answer: Testing in React is typically done using libraries like:

- **Jest** for unit testing.

- **React Testing Library** for testing components.
- **Enzyme** for testing component lifecycles and states.

```
// Simple Jest test for a component:
test('renders learn react link', () => {
  render(<App />);
  const linkElement = screen.getByText(/learn react/i);
  expect(linkElement).toBeInTheDocument();
});
```